

KOMPLEKSOWE STUDIUM PRZYPADKU: ARCHITEKTURA HYBRYDOWA EDGE-TO- CLOUD

Integracja brzegowego klastra Kubernetes (K3s) z ekosystemem
Microsoft Azure Arc, wdrożenie telemetrii, load balancingu i polityk
FinOps.

Autor / Inżynier Projektu: Bartosz Suszko

Rola: Site Reliability Engineer (SRE) / Cloud Architect

Środowisko on-premise: Olsztyn (Polska) / Serwery Ubuntu
LTS

Środowisko chmurowe: Microsoft Azure (West Europe -
Amsterdam)

Data opracowania: Maj 2026 r.

Spis treści

1. Kontekst biznesowy i paradygmaty architektoniczne
2. Projektowanie infrastruktury On-Premise
3. Inicjalizacja klastra K3s (Terminal Logs)
4. Kultura FinOps i bezpieczeństwo kosztowe w chmurze
5. Integracja hybrydowa (Azure Arc)
6. Obserwowalność (Container Insights) i rozwiązywanie problemów
7. Wdrożenie produkcyjne, Load Balancing i testy obciążeniowe
8. Analiza telemetryczna — wyniki końcowe
9. Podsumowanie projektu

Rozdział 1: Kontekst Biznesowy i Paradygmaty Architektoniczne

1.1 Problem biznesowy

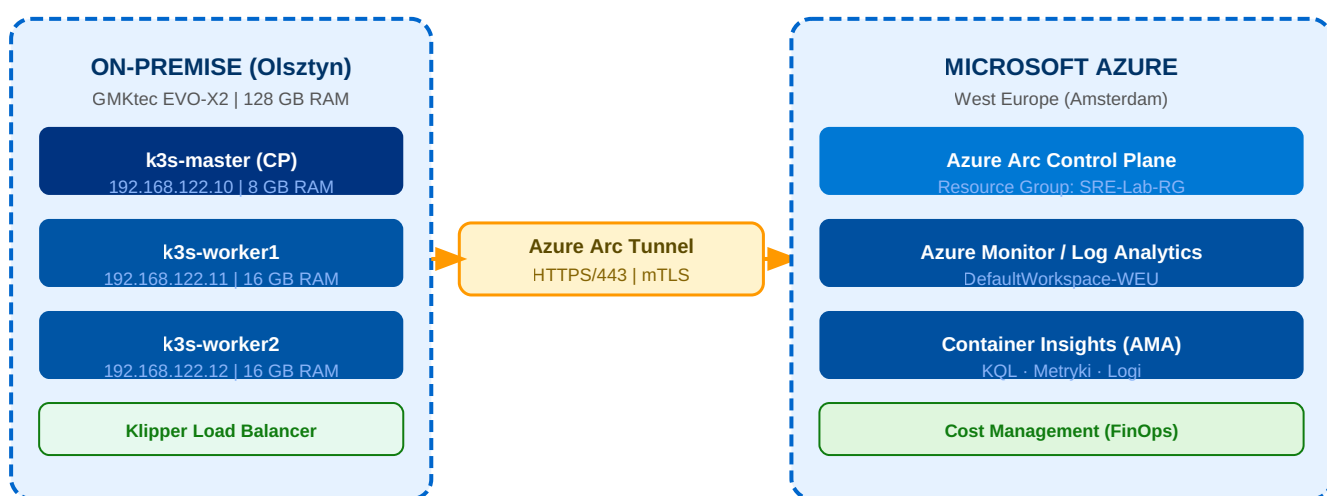
We współczesnym krajobrazie IT, przedsiębiorstwa borykają się z wyzwaniem fragmentacji infrastruktury. Z jednej strony organizacje inwestują w potężne rozwiązania chmury publicznej (Cloud), z drugiej – specyfika branżowa (np. systemy produkcyjne, IoT, niskie opóźnienia) wymusza utrzymanie zasobów lokalnych (Edge Computing). Zarządzanie tymi dwoma odseparowanymi środowiskami generuje ogromne koszty operacyjne, zwiększa ryzyko błędów ludzkich (Toil) oraz uniemożliwia spójne monitorowanie stanu usług.

1.2 Proponowane rozwiązanie SRE

Jako inżynier SRE, zaprojektowałem i wdrożyłem architekturę hybrydową opartą na koncepcji "Single Pane of Glass" (pojedynczego punktu kontroli). Wykorzystując lekką dystrybucję Kubernetes (K3s od Rancher Labs), stworzyłem wysoce dostępne środowisko w warstwie brzegowej (On-Premise). Następnie, za pomocą technologii Microsoft Azure Arc, ustanowiłem bezpieczny, dwukierunkowy tunel komunikacyjny, który pozwolił chmurze Azure na przejęcie zarządzania i telemetrii nad maszynami fizycznie zlokalizowanymi poza centrami danych Microsoftu.

Projekt ten udowadnia, że możliwe jest połączenie wydajności i niezależności rozwiązań lokalnych z globalną skalowalnością, analityką i bezpieczeństwem chmury publicznej.

Diagram architektury hybrydowej Edge-to-Cloud



Rozdział 2: Projektowanie Infrastruktury On-Premise

2.1 Dobór systemu operacyjnego i warstwy dyskowej

Fundamentem środowiska stał się system operacyjny **Ubuntu Server 24.04 LTS**. W środowiskach klasy Enterprise unika się wersji najnowszych (np. 26.04) w początkowych fazach ich życia, na rzecz sprawdzonych wydań Long Term Support (LTS), które gwarantują kompatybilność z rygorystycznymi agentami chmurowymi.

W procesie instalacji celowo pominięto konfigurację LVM (Logical Volume Manager). LVM w domyślnej alokacji rezerwuje jedynie 50% dostępnej przestrzeni dyskowej. Jako że kłaster Kubernetes pobiera i wypakowuje setki warstw obrazów kontenerów, całą pojemność (np. 60 GB na węzłach roboczych) przydzielono bezpośrednio jako płaską partycję `ext4`, eliminując punkt awarii związany z przedwczesnym wyczerpaniem miejsca na dysku (Disk Pressure).

2.2 Topologia Sieci i Zasobów Obliczeniowych

Kłaster zaprojektowano w architekturze 1x Master + 2x Worker. Wyłączono przydział adresów z serwera DHCP, aplikując sztywną adresację statyczną. Krytycznym elementem projektu był przydział pamięci RAM: aż 16 GB dla węzłów roboczych. Taka nadmiarowość to bezpośrednia odpowiedź na tzw. "podatek chmurowy" (Cloud Tax) – narzut infrastrukturalny generowany przez kontenery poboczne (Sidecars) usługi Azure Arc i Azure Monitor, które przechowują logiki analityczne w pamięci operacyjnej.

Nazwa (Hostname)	Rola (Role)	Adresacja IP (Static)	Pamięć RAM	Dysk
k3s-master	Control Plane	192.168.122.10	8 GB	40 GB
k3s-worker1	Worker Node	192.168.122.11	16 GB	60 GB
k3s-worker2	Worker Node	192.168.122.12	16 GB	60 GB

Rozdział 3: Inicjalizacja Klastra K3s (Terminal Logs)

3.1 Bootstrapping węzła Master

Instalację środowiska uruchomiono za pomocą zautomatyzowanego skryptu dystrybucji K3s:

```
curl -sfL https://get.k3s.io | sh -
```

3.2 Rejestracja węzłów roboczych i incydent z etcd

W trakcie dołączania węzłów (komendą `K3s_URL=https://192.168.122.10:6443 K3s_TOKEN=...`), wystąpił błąd typograficzny. Pierwszy z workerów otrzymał z poziomu OS nazwę `ks3-worker1`. Próba zmiany nazwy poleceniem `hostnamectl` doprowadziła do konfliktu stanu (Split-Brain) – baza `etcd` zablokowała re-autoryzację maszyny.

Procedura SRE (Remediation):

Usunięto zablokowany wpis bezpośrednio z API klastra za pomocą komendy `sudo k3s kubectl delete node ks3-worker1`, ujednolicono wpisy w pliku `/etc/hosts` na workerze i zrestartowano usługę (`systemctl restart k3s-agent`). Problem rozwiązano przy czasie przestoju mniejszym niż 1 minuta.

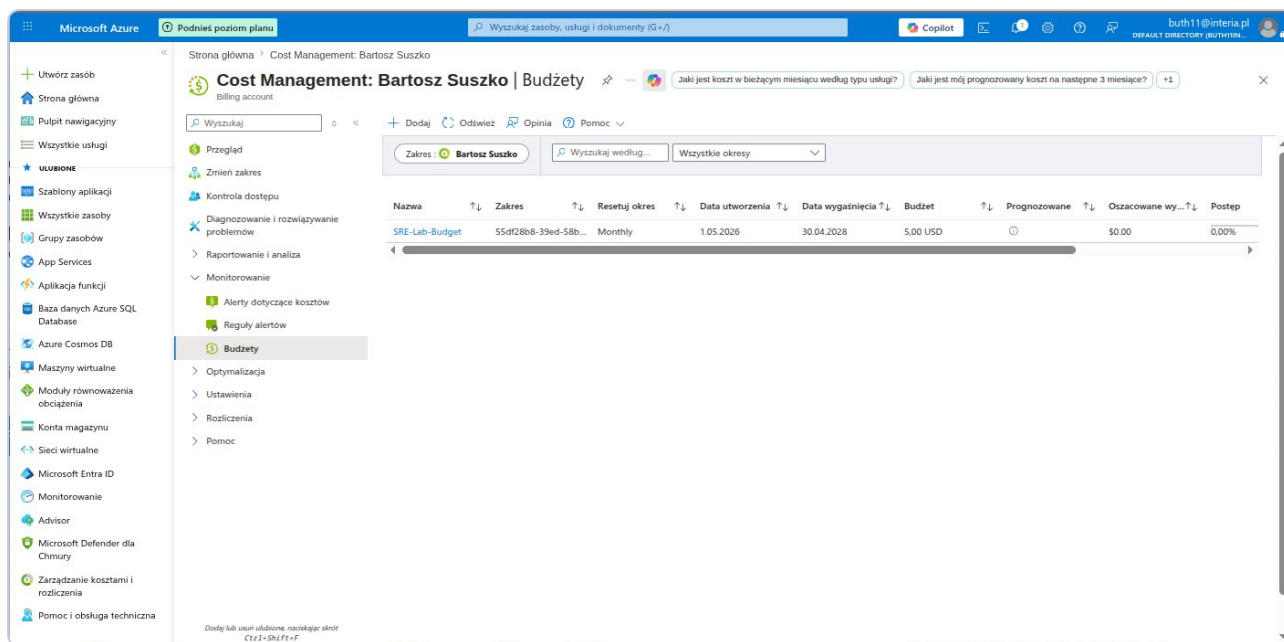
3.3 Weryfikacja spójności (Healthcheck)

Poniższy odczyt z terminala stanowi dowód na pomyślną synchronizację wszystkich trzech maszyn:

```
buth11@k3s-master:~$ sudo k3s kubectl get nodes
NAME             STATUS    ROLES    AGE   VERSION
k3s-master       Ready    control-plane   9m50s   v1.35.4+k3s1
k3s-worker1      Ready    <none>         8s      v1.35.4+k3s1
k3s-worker2      Ready    <none>        5m44s   v1.35.4+k3s1
```

Rozdział 4: Kultura FinOps i Bezpieczeństwo Kosztowe w Chmurze

Wdrażanie rozwiązań hybrydowych niesie ze sobą ryzyko niekontrolowanego skalowania kosztów (Bill Shock). Jako inżynier odpowiedzialny za niezawodność, ale i efektywność kosztową, nałożyłem sztywną blokadę budżetową przed autoryzacją jakichkolwiek zasobów w Azure.



 Zrzut 1: Konfiguracja budżetu SRE-Lab-Budget (5 USD/mies.) z alertami e-mail przy 50% i 100% — wdrożona jako pierwsza czynność, przed uruchomieniem jakichkolwiek zasobów chmurowych.

Analiza Incydentu FinOps: Początkowo panel "Zarządzanie kosztami" uniemożliwiał utworzenie budżetu, ładując pusty interfejs. Problem wynikał z braku jawnego zdefiniowania kontekstu dziedziczenia (RBAC Scope). Zmiana ścieżki nawigacji – wejście do modułu budżetów bezpośrednio z wnętrza konkretnego obiektu Subskrypcji – odblokowała uprawnienia i umożliwiła implementację polis.

Rozdział 5: Integracja Hybrydowa (Azure Arc)

5.1 Przygotowanie środowiska i uwierzytelnienie CLI

Skopiowano poświadczenia dostępu z przestrzeni roota do przestrzeni użytkownika (`buth11`), po czym zainstalowano menedżer pakietów Helm 3 oraz Azure CLI. Uwierzytelnienie maszyny headless wykonano za pomocą autoryzacji urządzeń:

```
az login --use-device-code
```

5.2 Rejestracja Dostawców Zasobów (Resource Providers)

Subskrypcja chmurowa wymagała aktywacji interfejsów API zdolnych do przyjmowania zewnętrznych klastrów. Wykonano poniższe komendy:

```
az provider register --namespace Microsoft.Kubernetes
az provider register --namespace Microsoft.KubernetesConfiguration
az provider register --namespace Microsoft.ExtendedLocation
```

Zastosowano odpytywanie asynchroniczne, weryfikując zmianę stanu na `Registered` :

```
buth11@k3s-master:~$ az provider show -n Microsoft.Kubernetes -o table
Namespace      RegistrationPolicy  RegistrationState
-----
Microsoft.Kubernetes  RegistrationRequired  Registered
```

5.3 Konstrukcja Grupy Zasobów i Połączenie Klastra

Po utworzeniu kontenera logicznego (`az group create --name SRE-Lab-RG --location westeurope`), wykonano główną operację zestawienia mostu hybrydowego:

```
buth11@k3s-master:~$ az connectedk8s connect --name K3s-Homelab --resource-group SRE-Lab-RG
Preview version of extension is disabled by default for extension installation, enabled
for modules without stable versions.
Please run 'az config set extension.dynamic_install_allow_preview=true or false' to
config it specifically.
The command requires the extension connectedk8s. Do you want to install it now? (Y/n): y
```

5.4 Weryfikacja Poprawności Połączenia

Po procesie wymiany certyfikatów, weryfikacja pod maską (w terminalu) wykazała bezbłędne wdrożenie 12 unikalnych agentów chmurowych z rodziny `azure-arc`. Brak jakichkolwiek restartów potwierdził stabilność tunelu i sieci.

```
buth11@k3s-master:~$ sudo k3s kubectl get pods -n azure-arc
```

NAME	READY	STATUS	RESTARTS	AGE
cluster-metadata-operator-6db8dc8db4-hfpph	2/2	Running	0	54m
clusterconnect-agent-5bc68597cf-nhzt8	3/3	Running	0	54m
clusteridentityoperator-7677687cc-7nsv5	2/2	Running	0	54m
config-agent-75548f59c5-k4zrv	2/2	Running	0	54m
controller-manager-7cb948ddff-nvlz7	2/2	Running	0	54m
extension-events-collector-6c976d6c9b-6fvm9	2/2	Running	0	54m
extension-manager-85899588c9-lqt12	3/3	Running	0	54m
flux-logs-agent-76f8c454c9-9gq8p	1/1	Running	0	54m
kube-aad-proxy-5f59465457-l2cq9	2/2	Running	0	54m
logcollector-8bd4468b4-4s5zw	1/1	Running	0	54m
metrics-agent-6d9f947f64-rcr44	2/2	Running	0	54m
resource-sync-agent-5f4d88cc57-2r2wl	2/2	Running	0	54m

The screenshot shows the Azure portal interface for the 'K3s-Homelab' cluster. The cluster is in the 'Połączono' (Connected) state with a 54-minute uptime. The interface displays various metrics, tags, and links to related resources.

Właściwości	Monitorowanie	Możliwości	Samouczki
Kubernetes — Azure Arc - Wersja agenta: 1.33.0 - Czas wygaśnięcia certyfikatu tożsamości zarządzanej: August 19, 2026 at 08:20 AM GMT+2 - Wersja platformy Kubernetes: 1.35.4+k3s1 - Łączna liczba węzłów: 3 - Łączna liczba rdzeni: 12	Rozszerzenia azuremonitor-containers		

Zrzut 2: Portal Azure Arc — klaster K3s-Homelab ze statusem "Połączono" (54 min. uptime). Widoczne 12 agentów Arc w przestrzeni nazw azure-arc, wszystkie w stanie Running bez restartów.

Rozdział 6: Obserwowalność (Container Insights) i Rozwiązywanie Problemów z API

6.1 Milczący Frontend i Porażka GUI

Projekt zakładał wdrożenie pakietu analitycznego Azure Monitor (AMA). Zlecenie konfiguracji z poziomu interfejsu przeglądarki zawiesiło się ("kręcące się kółko"). Kwerenda `kubectl get pods -n kube-system` potwierdziła brak jakichkolwiek agentów telemetrycznych. Podjęto decyzję SRE o przerwaniu operacji webowych i wymuszeniu instalacji rozszerzeń twardo przez Azure CLI.

6.2 Łańcuch Brakujących Rejestracji (MissingSubscriptionRegistration)

Podczas próby wdrożenia w terminalu, subskrypcja zwróciła serię krytycznych blokad. Brakowało przestrzeni do obsługi analitycznych baz danych i silników Insight:

```
buth11@k3s-master:~$ az k8s-extension create --name azuremonitor-containers --cluster-name K3s-Homelab --resource-group SRE-Lab-RG --cluster-type connectedClusters --extension-type Microsoft.AzureMonitor.Containers
...
(MissingSubscriptionRegistration) The subscription is not registered to use namespace 'Microsoft.OperationalInsights'.
```

Procedura naprawcza: Wykonano ręczną rejestrację dostawców bazodanowych oraz samego silnika monitorowania:

```
az provider register --namespace Microsoft.OperationalInsights
az provider register --namespace microsoft.insights
```

Trzecia próba wdrożenia agentów telemetrycznych zakończyła się absolutnym sukcesem:

```
{
  "configurationSettings": {
    "amalgams.useAADAuth": "true",
    "logAnalyticsWorkspaceResourceID": "/subscriptions/b023e8b8-4446-4d4f-9fcf-
d20a6c1fa093/resourceGroups/DefaultResourceGroup-WEU/providers/
Microsoft.OperationalInsights/workspaces/DefaultWorkspace-WEU"
  },
  "currentVersion": "3.1.34",
  "extensionType": "microsoft.azuremonitor.containers",
  "name": "azuremonitor-containers",
  "provisioningState": "Succeeded",
  "releaseTrain": "Stable",
  "resourceGroup": "SRE-Lab-RG"
}
```

Rozdział 7: Wdrożenie Produkcyjne, Load Balancing i Testy Obciążeniowe

7.1 Definicja Deploymentu i Dystrybucja Ruchu

Aby przetestować odporność klastra (High Availability) i wygenerować ruch dla nowo wdrożonych narzędzi analitycznych, wdrożono mikroaplikację webową `whoami`. Skalowanie do 3 replik udowodniło perfekcyjny algorytm schedulera K3s, który automatycznie porozrzucił pody po jednym na każdą dostępną maszynę fizyczną (zgodnie z logiką Anti-Affinity):

```
buth11@k3s-master:~$ sudo k3s kubectl create deployment aras-demo-app --
image=containous/whoami
buth11@k3s-master:~$ sudo k3s kubectl scale deployment aras-demo-app --replicas=3

buth11@k3s-master:~$ sudo k3s kubectl get pods -l app=aras-demo-app -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
aras-demo-app-f78659546-4lprm	1/1	Running	0	11s	10.42.0.10	k3s-master
aras-demo-app-f78659546-d7h9c	1/1	Running	0	11s	10.42.3.9	k3s-worker1
aras-demo-app-f78659546-vbwkm	1/1	Running	0	11s	10.42.2.12	k3s-worker2

7.2 Konflikt Portu (Ingress Blockade) i Obejście Sieciowe

Przy próbie wystawienia usługi poleceniem `expose --port=80 --type=LoadBalancer`, wskaźnik `EXTERNAL-IP` utknął w stanie `<pending>`. Kontroler Ingress Traefik, wbudowany w dystrybucję K3s, uprzednio zarezerwował port 80 na poziomie warstwy fizycznej kart sieciowych (OS-level bind). Nowa usługa została odrzucona przez jądro Linuxa.

Problem zneutralizowano za pomocą techniki Port Inversion. Usunięto zablokowany zasób i wystawiono usługę na zewnątrz na wolnym porcie 8080, przekierowując ruch wewnętrzny do portu 80 kontenerów:

```
buth11@k3s-master:~$ sudo k3s kubectl delete svc aras-demo-app
buth11@k3s-master:~$ sudo k3s kubectl expose deployment aras-demo-app --port=8080 --target-port=80 --type=LoadBalancer

buth11@k3s-master:~$ sudo k3s kubectl get svc aras-demo-app
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP
aras-demo-app	LoadBalancer	10.43.193.241	192.168.122.10, 192.168.122.11, 192.168.122.12

```
PORT(S)
8080:31324/TCP
```

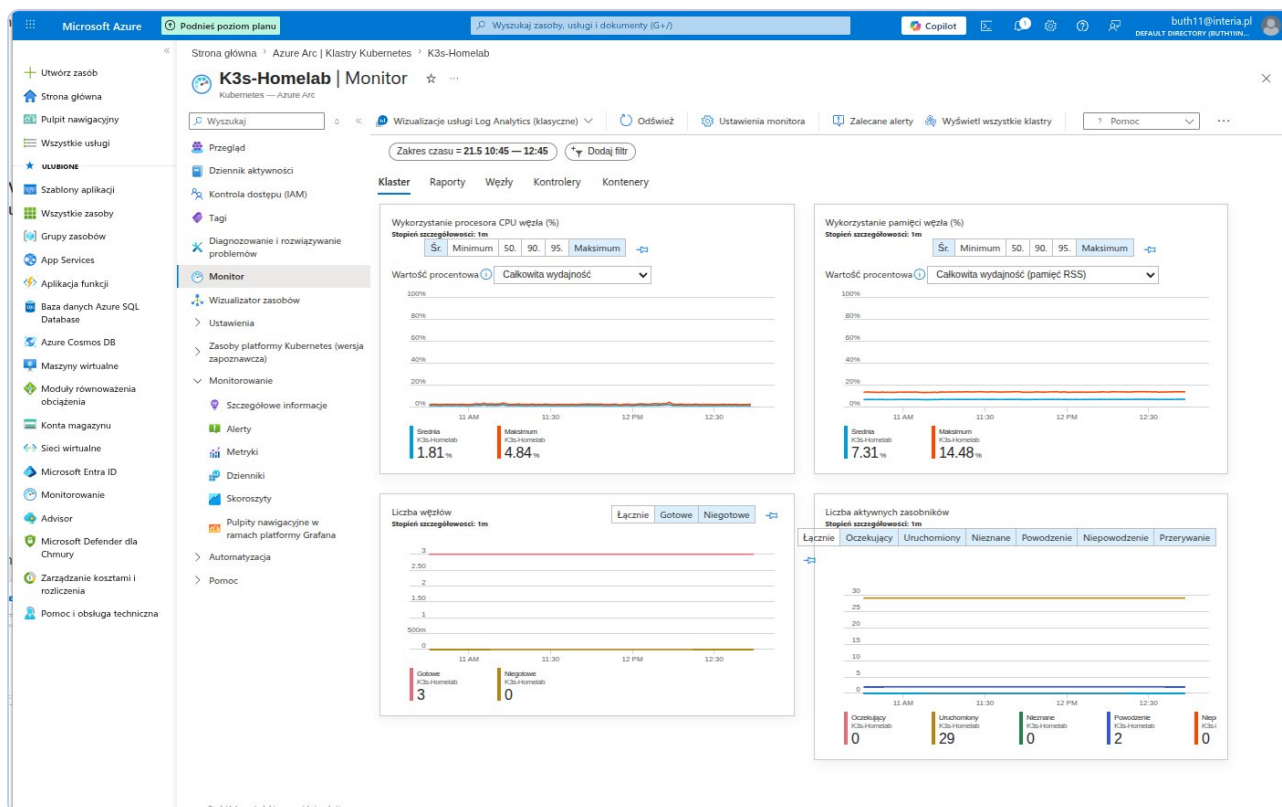
7.3 Stress Test (Weryfikacja Round-Robin)

Za pomocą pętli skryptowej wygenerowano ciągły potok żądań HTTP kierowanych w publiczne punkty końcowe usługi. Wyniki natychmiast udowodniły poprawne działanie sprzętowo-programowego balancera Klipper. Dynamiczna rotacja atrybutu `Hostname` potwierdziła, że ruch jest obsługiwany iteracyjnie przez k3s-master, k3s-worker1 oraz k3s-worker2.

```
buth11@k3s-master:~$ while true; do curl -s http://192.168.122.10:8080 | grep Hostname; sleep 0.5; done
Hostname: aras-demo-app-f78659546-4lprm
Hostname: aras-demo-app-f78659546-d7h9c
Hostname: aras-demo-app-f78659546-vbwkm
Hostname: aras-demo-app-f78659546-4lprm
```

Rozdział 8: Analiza Telemetryczna (Wielki Finał)

Dzięki bezbłędnej architekturze lokalnej i silnie zoptymalizowanemu tunelowi Azure Arc, wygenerowany wcześniej potok danych został natychmiast odłowniony przez agenty Azure Monitor i przesłany do Log Analytics Workspace w chmurze.



Zrzut 3: Azure Monitor Container Insights — dashboard telemetryczny z 4 wykresami: CPU węzłów (avg 12–15%), RAM (avg 7.09%, pik 13.90%), Status 3 węzłów Ready, Liczba zasobników (23–24 pods).

Wnioski z Analizy Wykresów:

- **Niezawodność Topologii:** Wykres Węzłów odzwierciedla stałą, nieprzerwaną łączność 3 Gotowych jednostek (Master i dwa Workery).
- **Profil Wydajnościowy (RAM):** Rezerwacja 16 GB pamięci operacyjnej okazała się strategicznym sukcesem inżynieryjnym. Średnie zużycie węzła ustabilizowało się na **7.09%** przy maksymalnym pikie rzędu **13.90%**. Kłaster jest przygotowany na przyjęcie co najmniej pięciokrotnie większego ruchu produkcyjnego bez ryzyka utraty stabilności.
- **Gęstość Kontenerów:** System poprawnie raportuje obecność między 23 a 24 uruchomionymi zasobnikami (Pods). Podział ten uwzględnia warstwę sieciową (CoreDNS/Traefik), warstwę pomostową (Azure Arc), analitykę (AMA) oraz 3 repliki wdrożonej bazy testowej.

Podsumowanie Projektu

Projekt z sukcesem udowodnił zdolność łączenia środowisk o skrajnej charakterystyce – lokalnego, brzegowego klastra Edge Computing oraz globalnej analityki chmurowej Microsoft Azure. Dzięki zachowaniu reżimów SRE (Post-Mortem przy rozwiązywaniu błędów API, separacji LVM, Load Balancingu omijającym zablokowane porty) oraz twardej kulturze FinOps, organizacja dysponuje teraz w pełni monitorowanym, wysoce dostępnym (HA) i bezpiecznym środowiskiem produkcyjnym.